



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

---

# درس شبکه عصبی

# پاسخ تمرین شماره پنجم

---


سوال اول : علی ممتحن - ۸۱۰۱۰۴۳۲۱

سوال دوم : سید احمد رکنی حسینی - ۸۱۰۱۰۴۳۰۶

# فهرست مطالب

**فهرست تصاویر** ۴

**فهرست جداول** ۵

**۱ سوال یک** ۶

۱—۱ آماده‌سازی داده‌ها . . . . . ۶

۱—۱—۱ بررسی اولیه داده‌ها . . . . . ۶

۱—۱—۲ حذف کلاس‌های کم‌نمونه . . . . . ۷

۱—۱—۳ نرمال‌سازی و تغییر اندازه تصاویر . . . . . ۷

۱—۱—۴ افزایش داده (*Data Augmentation*) . . . . . ۸

۱—۲ تقسیم دادگان . . . . . ۹

۱—۲—۱ روش *Graph Sampling* در مقاله . . . . . ۹

۱—۲—۲ تقسیم داده‌ها در این تمرین . . . . . ۱۰

۱—۳ آموزش مدل *ResNet* . . . . . ۱۰

۱—۴ آموزش مدل *BotNet* . . . . . ۱۱

۱—۵ برسی نتایج *BotNet* . . . . . ۱۲

۱—۶ تحلیل نتایج . . . . . ۱۴

۱—۶—۱ مقایسه عملکرد مدل‌های ResNet۵۰ و BoTNet۵۰ . . . . . ۱۴

۱—۶—۲ شرایطی که مدل عملکرد ضعیف‌تری نشان داده است . . . . . ۱۵

۱—۶—۳ . . . . . ۱۵

**۲ سوال دو** ۱۶

۲—۱ سوال‌های تئوری . . . . . ۱۶

۲—۱—۱ تفاوت *autoregressive* و *masked iterative generation* . . . . . ۱۶

۲—۱—۲ *forward masking* به‌عنوان *noise model* . . . . . ۱۶

۲—۱—۳ چرا *loss* باید *reweight* شود . . . . . ۱۷

۲–۲ داده، پرامپت و ابزارهای ارزیابی . . . . . ۱۸
۲–۲–۱ ساخت *pipeline* داده . . . . . ۱۸
۲–۲–۲ طراحی *prompt* در قالب *chat format* . . . . . ۱۸
۲–۲–۳ *SQL normalization* و معیارها . . . . . ۱۹
۲–۳ بارگذاری مدل و آموزش . . . . . ۱۹
۲–۳–۱ بارگذاری مدل و مفاهیم جدید . . . . . ۱۹
۲–۳–۲ پیاده‌سازی *forward masking* . . . . . ۲۰
۲–۳–۳ مرز *prompt/answer* و ماسک‌کردن پاسخ . . . . . ۲۰
۲–۳–۴ *training loop* و *loss* . . . . . ۲۱
۲–۴ تولید متن و ارزیابی . . . . . ۲۱
۲–۴–۱ پیاده‌سازی *block diffusion sampling* . . . . . ۲۱
۲–۴–۲ *post-processing* برای استخراج SQL . . . . . ۲۲
۲–۴–۳ *evaluation* و تحلیل خطا . . . . . ۲۲
۲–۴–۴ پنج ایده برای بهبود . . . . . ۲۳

# فهرست تصاویر


۱ نمونه‌ای از تصاویر موجود در دیتاست . . . . . ۶
۲ توزیع تعداد تصاویر در هر کلاس قبل از پیش‌پردازش . . . . . ۷
۳ توزیع تعداد تصاویر در هر کلاس بعد از پیش‌پردازش . . . . . ۹
۴ نمودار خطا و دقت مدل *ResNet* هنگام آموزش . . . . . ۱۱
۵ نمودار خطا و دقت مدل *BotNet* هنگام آموزش . . . . . ۱۲
۶ خروجی هیت مپ لایه توجه برای چند نمونه . . . . . ۱۳
۷ مقایسه شهودی تولید *autoregressive* و *masked iterative* . . . . . ۱۷

# فهرست جداول

۱ اندازه داده‌های استفاده‌شده در این پیاده‌سازی . . . . . ۱۸

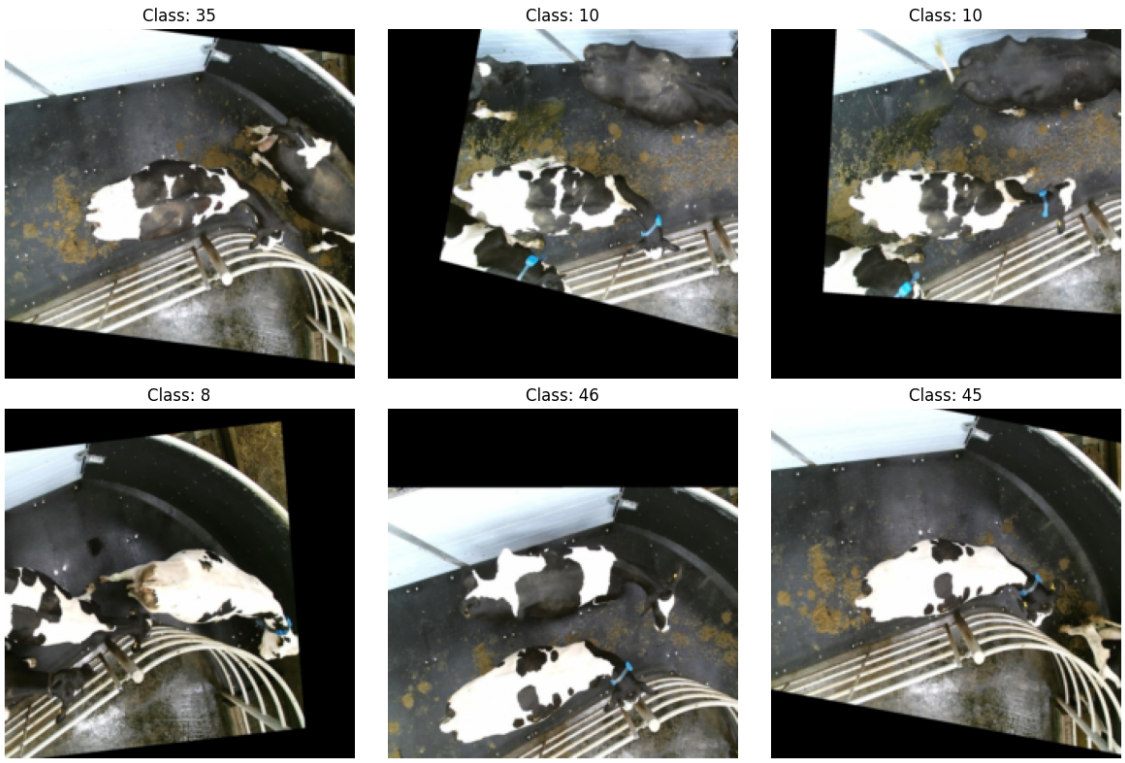
# ۱ سوال یک

## ۱–۱ آماده‌سازی داده‌ها

### ۱–۱–۱ بررسی اولیه داده‌ها

در این پروژه از دیتاست مربوط به مقاله *Identification Individual Cow for Attention Counterfactual and Sampling Graph with Model Transformer BottleNet Optimized* استفاده شد. این داده‌ها شامل تصاویر مختلف از گاوهای متفاوت برای مسئله بازشناسایی (*Re-identification*) هستند.

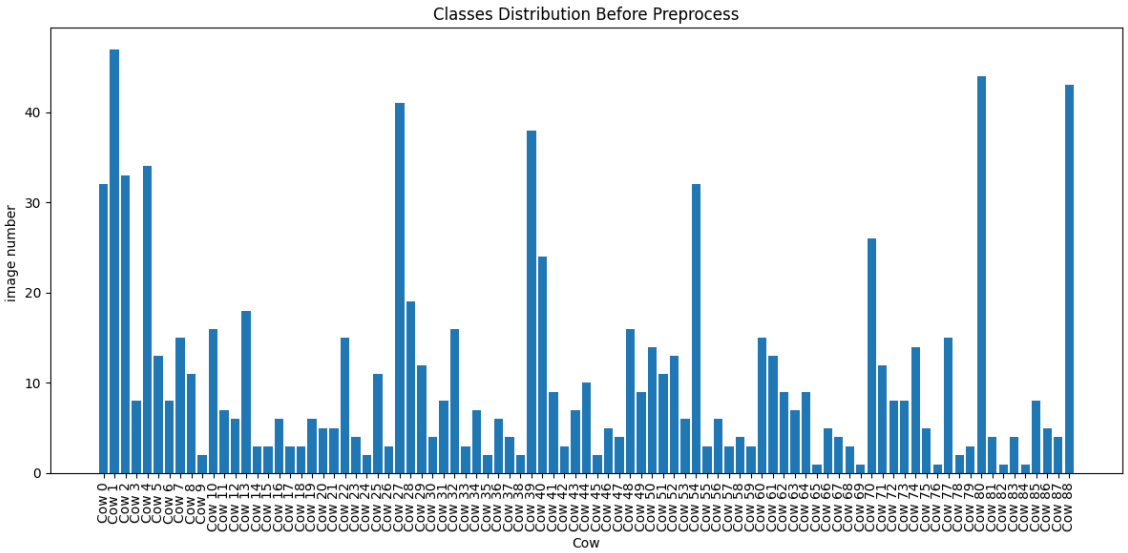
در ابتدا چند نمونه از تصاویر برای بررسی کیفیت، زاویه دید و شرایط نوری نمایش داده شد. که در شکل ۱ مشاهده می شود.




شکل ۱: نمونه‌ای از تصاویر موجود در دیتاست

با بررسی نمودار توزیع کلاس‌ها مشاهده شد که داده‌ها دارای عدم توازن هستند؛ به این صورت که برخی

کلاس‌ها تعداد تصاویر زیادی دارند اما برخی کلاس‌ها بسیار کم‌نمونه هستند.




شکل ۲: توزیع تعداد تصاویر در هر کلاس قبل از پیش‌پردازش

این موضوع می‌تواند باعث شود مدل به سمت کلاس‌های پرتعداد متمایل شود و عملکرد ضعیفی روی کلاس‌های کم‌نمونه داشته باشد.

### ۱–۱–۲ حذف کلاس‌های کم‌نمونه

کلاس‌هایی که کمتر از ۵ تصویر داشتند حذف شدند. دلیل این کار:

- جلوگیری از بیش‌برازش (*Overfitting*)
- ایجاد پایداری بیشتر در آموزش

### ۱–۱–۳ نرمال‌سازی و تغییر اندازه تصاویر

تمام تصاویر به اندازه $224 \times 224$ تغییر اندازه داده شدند تا با ورودی مدل‌های مبتنی بر *Transformer* سازگار باشند.

همچنین تصاویر با استفاده از میانگین و انحراف معیار *ImageNet* نرمال‌سازی شدند. نرمال‌سازی باعث می‌شود:

- سرعت همگرایی افزایش یابد
- آموزش پایدارتر شود
- سازگاری با مدل‌های از پیش آموزش‌دیده بهتر شود

### ۱–۱–۴ افزایش داده (*Data Augmentation*)

برای افزایش دقت مدل و جلوگیری از بیش‌برازش، از تکنیک‌های مختلف افزایش داده استفاده شد که فقط در مرحله آموزش اعمال شدند.

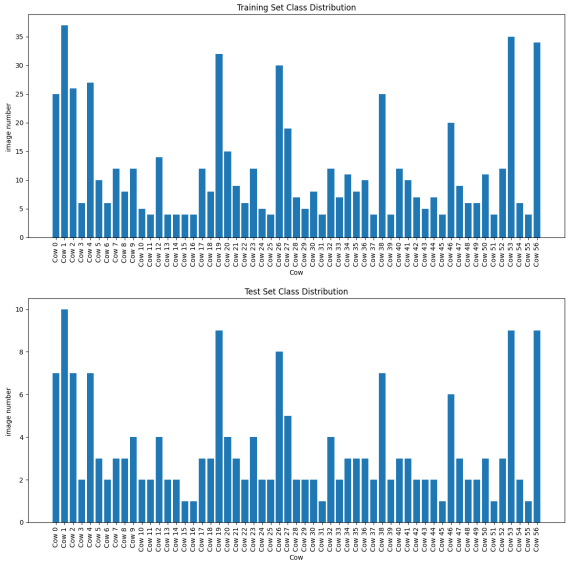
مهم‌ترین آن‌ها عبارتند از:

- *RandomResizedCrop*: جلوگیری از وابستگی مدل به مکان دقیق سوژه
- *HorizontalFlip*: افزایش تنوع زاویه دید
- *Rotation* و *Affine*: شبیه‌سازی تغییر زاویه دوربین
- *ColorJitter*: مقاوم‌سازی نسبت به تغییرات نور
- *GaussianBlur*: افزایش تحمل نسبت به تاری تصویر
- *RandomErasing*: جلوگیری از تمرکز مدل روی یک ناحیه خاص (مهم در مسائل *Re-ID*)

استفاده از *RandomErasing* در بازشناسایی اهمیت زیادی دارد، زیرا باعث می‌شود مدل به‌جای تمرکز روی یک لکه یا ناحیه خاص، ویژگی‌های کلی‌تری از گاو را یاد بگیرد. با استفاده از روش های گفته شده تعداد داده های آموزش چند برابر شد. اینکار باعث افزایش دقت مدل ها نیز شد.

در داده‌های آزمون فقط تغییر اندازه و نرمال‌سازی انجام شد و هیچ افزایش داده‌ای اعمال نشد.

در شکل ۳ توزیع کلاس ها در داده آموزش و تست بعد از پیش پردازش نشان داده شده است. تعداد کلاس ها به ۵۷ کاهش پیدا کرده است. و تعداد داده تست برابر ۱۹۴ و داده آموزش برابر ۶۵۷ است. البته این مقادیر بدون آگمنت داده است. و بعد از آگمنت تعداد داده آموزش چند برابر شده است.




شکل ۳: توزیع تعداد تصاویر در هر کلاس بعد از پیش‌پردازش

## ۱–۲ تقسیم دادگان

### ۱–۲–۱ روش *Graph Sampling* در مقاله

در مقاله از روش *Graph Sampling* برای تقسیم داده‌ها استفاده شده است. در این روش:

- هر تصویر یک گره (*Node*) در گراف در نظر گرفته می‌شود.
- شباهت بین تصاویر به صورت یال (*Edge*) مدل می‌شود.

سپس با نمونه‌برداری از گراف، داده‌ها به شکلی تقسیم می‌شوند که شباهت زیاد بین تصاویر آموزش و آزمون کنترل شود.

این روش در مسئله بازشناسایی گاو مفید است زیرا:

- ممکن است چند تصویر تقریباً مشابه از یک گاو وجود داشته باشد.
- تقسیم تصادفی ساده ممکن است نمونه‌های بسیار مشابه را در آموزش و آزمون قرار دهد.

در این حالت مدل عملکرد غیرواقعی و بیش از حد خوش‌بینانه خواهد داشت. اما *Graph Sampling* باعث می‌شود ارزیابی واقعی‌تر باشد و مدل مجبور شود ویژگی‌های تعمیم‌پذیرتری یاد بگیرد.

### ۱–۲–۲ تقسیم داده‌ها در این تمرین

در این پروژه از تقسیم ساده تصادفی استفاده شد و داده‌ها با نسبت ۸۰/۲۰ به آموزش و آزمون تقسیم شدند. و همانطور که خواسته شده بود از روش مقاله استفاهده نشده است.

- ۸۰ درصد داده‌ها برای آموزش
- ۲۰ درصد داده‌ها برای آزمون

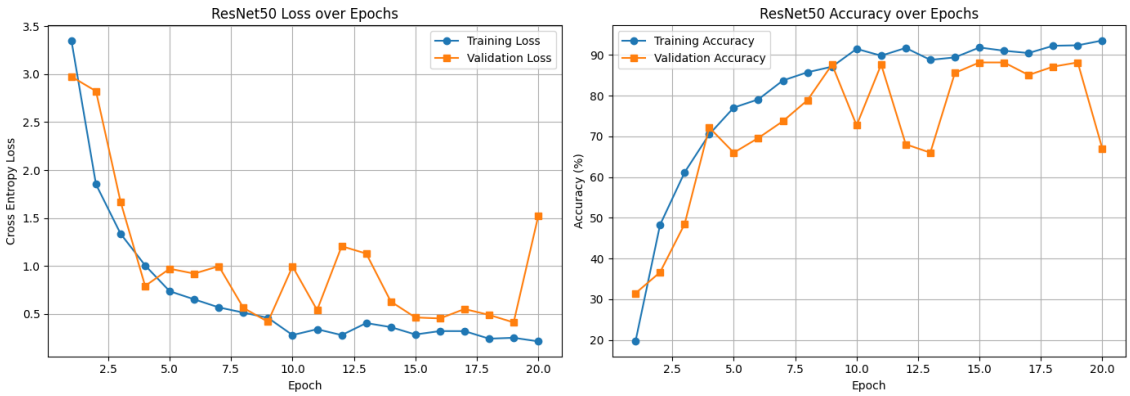
تمام کلاس‌های باقی‌مانده در هر دو بخش حضور دارند.

اندازه *Batch Size* برابر ۳۲ انتخاب شد که تعادل مناسبی بین مصرف حافظه و پایداری آموزش ایجاد می‌کند. و در صورت سوال نیز خواسته شده بود.

## ۱–۳ آموزش مدل *ResNet*

**ResNet** *Residual Network* یک معماری شبکه‌های کانولوشنی است که با معرفی *Skip Connection* اتصال میانبر مشکل ناپدید شدن گرادیان در شبکه‌های بسیار عمیق را حل کرد؛ به‌جای یادگیری مستقیم تابع خروجی $H(x)$، شبکه فقط باقی‌مانده $F(x) = H(x) - x$ را یاد می‌گیرد و ورودی را مستقیم به خروجی اضافه می‌کند. ساختار اصلی آن شامل بلوک‌های باقی‌مانده است که در دو نوع *Basic* (دو لایه $3*3$) و *Bottleneck* (سه لایه $1*1$ و $3*3$) پیاده‌سازی می‌شود و مدل‌های معروف آن *ResNet-18*، *34*، *50*، *101* و *152* هستند. این اتصالات میانبر امکان ساخت شبکه‌های صدها لایه‌ای را بدون افت عملکرد فراهم کرد امروزه به‌عنوان یکی از پرکاربردترین *backbone*ها در طبقه‌بندی تصاویر، تشخیص اشیاء (*Faster/Mask R-CNN*)، تقسیم‌بندی معنایی، تشخیص پزشکی، خودروهای خودران و *transfer learning* استفاده می‌شود و هنوز پایه بسیاری از مدل‌های مدرن بینایی کامپیوتری است.

این مدل با استفاده از داده آماده شده با تعداد *epoch=20* ترین شده است. نمودار خطا و دقت هنگام فرآیند آموزش در شکل ۴ نشان داده شده است.



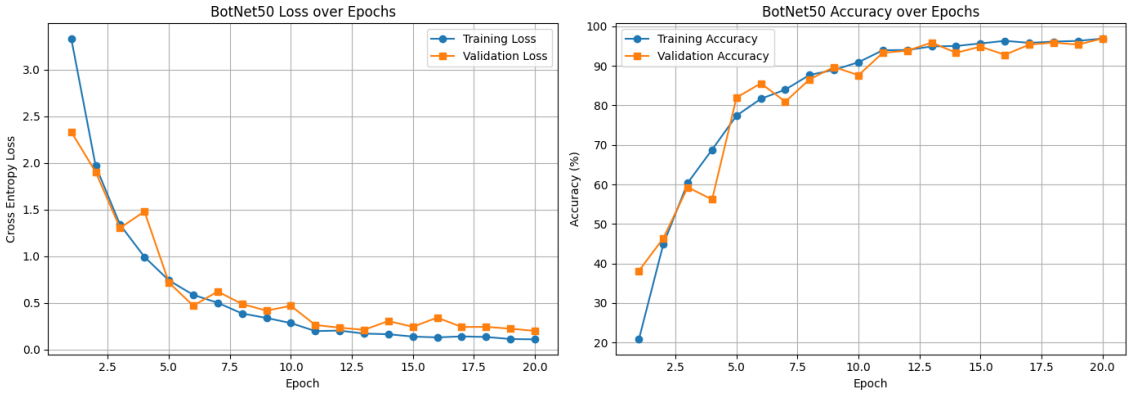

شکل ۴: نمودار خطا و دقت مدل *ResNet* هنگام آموزش

## ۱–۴ آموزش مدل *BotNet*

معماری *BoTNet* در مقاله یک مدل هیبریدی مبتنی بر *ResNet50* است که بعضی از لایه های آن را با *Muti Head Self Attention* جایگزین کرده است. در واقع ترکیب CNN و مکانیزم توجه است. این ساختار باعث می‌شود مدل بهتر الگوهای پیچیده پوست گاو را تشخیص دهد و وابستگی‌های بلندمدت را مدل کند. همچنین، با و Attention Counterfactual عملکرد را در برابر نویز پس‌زمینه بهبود می‌بخشد. در کل، این معماری دقت شناسایی فردی افزایش میدهد. .

در داخل فایل نوتبوک مربوط به این سوال معماری این مدل خروجی گرفته شده است.

فرآیند آموزش با همان دیتای آماده شده در بخش های قبلی روی این مدل نیز انجام شده است. که نمودار خطا و دفت در طول آموزش در شکل ۵ آمده است. که همانطور که دیده میشود دقت و خطا از مدل *ResNet* کمتر است.

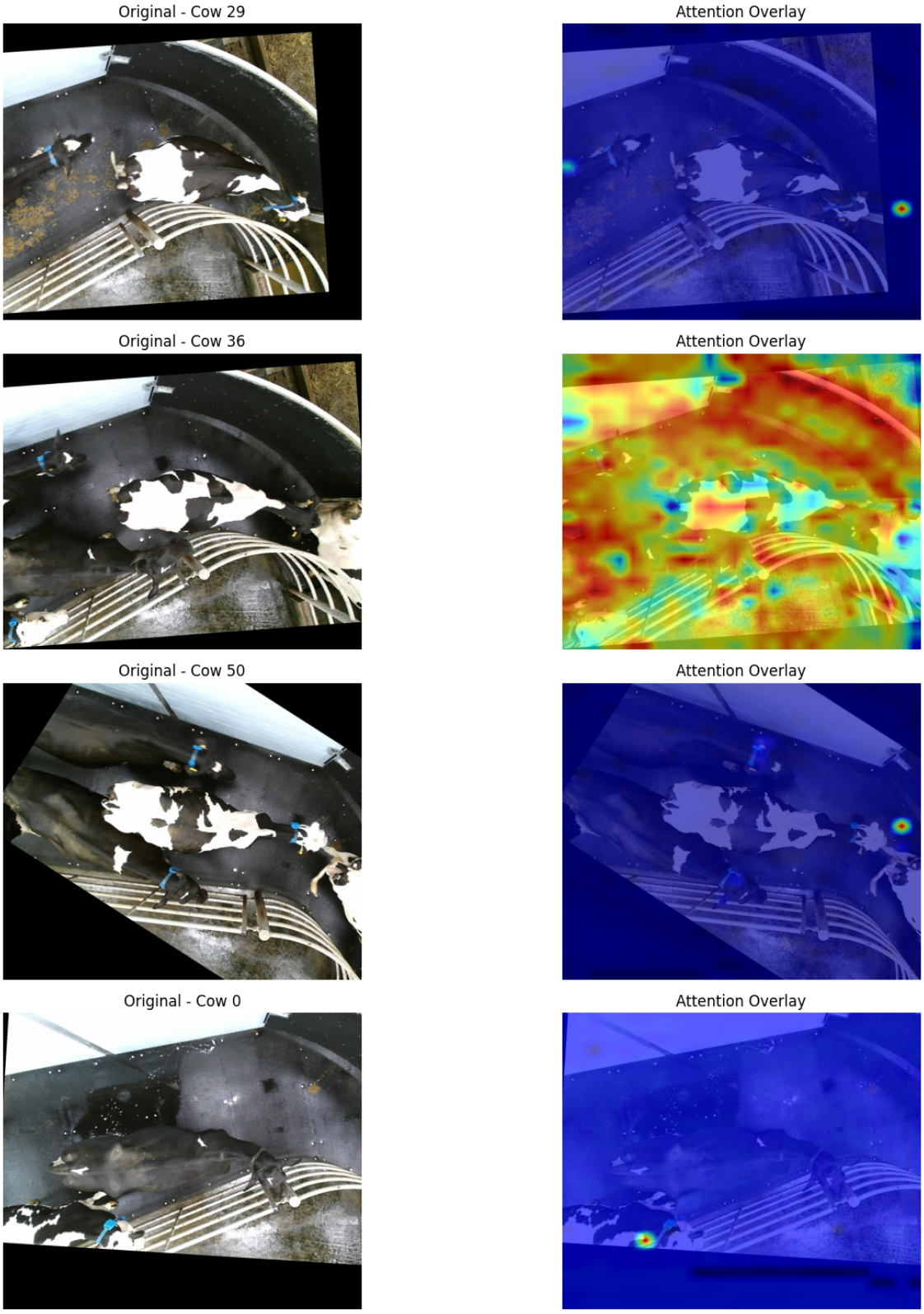



شکل ۵: نمودار خطا و دقت مدل *BotNet* هنگام آموزش

## ۵–۱ برسی نتایج *BotNet*

در شکل ۶ نمونه ای از خروجی هیتمپ آمده است.

## ۱ سوال یک




شکل ۶: خروجی هیت مپ لایه توجه برای چند نمونه

این طور که به نظر میرسد مدل در بعضی از مواقع روی قسمت خاص و کوچکی از تصویر تمرکز داشته است. اما همانطور که دیده می شود. در شکل دوم مکانیزم توجه روی کل محیط و محیط پس زمینه بوده است. در واقع با استفاده از اطلاعاتی طبقه بندی را انجام داده است که ربطی به کلاس بندی در شرایط واقعی ندارد. در شکل اول هم میبینیم که توجه اصلا به جای اشتباهی اشاره کرده است.

## ۱–۶ تحلیل نتایج

### ۱–۶–۱ مقایسه عملکرد مدل‌های ResNet۵۰ و BoTNet۵۰

با توجه به نتایج آموزش ۲۰ اپوک برای دو مدل، مقایسه عملکرد به شرح زیر است:

- **مدل ResNet۵۰**:
  - دقت آموزش نهایی: ٪۹۳/۵۳
  - دقت اعتبارسنجی نهایی: ٪۶۷/۰۱
  - بهترین دقت اعتبارسنجی در طول آموزش: ٪۸۸/۱۴ (اپوک ۱۵ و ۱۶)
  - افت قابل توجه دقت اعتبارسنجی در اپوک‌های پایانی (از حدود ٪۸۸ به ٪۶۷)
  - نشانه واضح **overfitting** شدید در نیمه دوم آموزش
- **مدل BoTNet۵۰**:
  - دقت آموزش نهایی: ٪۹۶/۸۸
  - دقت اعتبارسنجی نهایی: ٪۹۶/۹۱
  - بهترین دقت اعتبارسنجی: ٪۹۶/۹۱ (اپوک ۲۰)
  - روند بهبود تقریباً پایدار و بدون افت شدید در دقت اعتبارسنجی
  - اختلاف بسیار کم بین دقت آموزش و اعتبارسنجی ⁉ **overfitting بسیار کمتر**

در این دیتاست و با تنظیمات انجام‌شده، مدل **BoTNet۵۰ به طور واضح عملکرد بسیار بهتری** نسبت به ResNet۵۰ داشته است. این مدل نه تنها دقت بالاتری کسب کرده، بلکه تعمیم‌پذیری بسیار بهتری نشان

داده و و به نظر نشانه overfitting در آن کم است و دقت آموزش و ارزیابی همخوانی دارد. و نتایج کاملاً با انتظارات منطبق و حتی بهتر از انتظار اولیه است.

تنها نکته غیرمنتظره، عملکرد نسبتاً ضعیف و overfitting شدید مدل ResNet۵۰ بود که احتمالاً به دلیل عدم استفاده از تکنیک‌های regularization قوی‌تر (مانند label smoothing، mixup یا قوی‌تر کردن (augmentation رخ داده است.

### ۱–۶–۲ شرایطی که مدل عملکرد ضعیف‌تری نشان داده است

عملکرد ضعیف (به ویژه در مدل (ResNet۵۰ عمدتاً در شرایط زیر مشاهده شد:

۱. **عدم تعادل کلاس‌ها و تعداد کم نمونه در برخی کلاس‌ها:**

- کلاس‌هایی با تعداد تصاویر بسیار کم باعث overfitting شدید روی کلاس‌های پرتعداد می‌شوند.
- ResNet۵۰ به دلیل ساختار محلی (local receptive field) بیشتر تحت تأثیر این عدم تعادل قرار می‌گیرد.

۲. **حساسیت کمتر BoTNet۵۰ به این مشکلات:**

- مکانیزم توجه (attention) به طور طبیعی می‌تواند اطلاعات نامرتبط را نادیده بگیرد و بر نواحی مهم تمرکز کند [؟] همین ویژگی باعث مقاومت بیشتر BoTNet۵۰ در برابر عدم تعادل کلاس و overfitting شده است.

اگر دادگان در کلاس های مختلف توازن کافی برخوردار بودند به نظر مدل ResNet میتوانست عملکرد بهتری نشان دهد.

### ۱–۶–۳

بله به دلیل عدم داده کافی مدل نمیتواند ویژگی های کلاس را به خوبی پیدا کند برای مثال شکل اول هیتمپ نمونه ای از کلاسی است که تعداد داده آن بسیار کم است.

# ۲ سوال دو

## ۲–۱ سوال‌های تئوری

### ۲–۱–۱ تفاوت *autoregressive* و *masked iterative generation*

در روش *autoregressive* تولید متن به‌صورت چپ‌به‌راست انجام می‌شود؛ یعنی مدل در هر گام فقط توکن بعدی را با شرط روی توکن‌های قبلی پیش‌بینی می‌کند. بنابراین اگر اوایل تولید یک انتخاب اشتباه انجام شود، همان خطا به‌صورت زنجیره‌ای به ادامه خروجی منتقل می‌شود.

در مقابل، در رویکرد *masked prediction* مدل هنگام آموزش یاد می‌گیرد در یک توالی که بخشی از آن با `[MASK]` پنهان شده، توکن‌های پنهان را بازسازی کند. سپس در *masked iterative generation* خروجی از یک پاسخ کاملاً ماسک‌شده شروع می‌شود و در چند گام، بخشی از ماسک‌ها به‌تدریج پر می‌شوند تا پاسخ نهایی شکل بگیرد.

مزیت اصلی روش *masked iterative* این است که امکان «بازنگری تدریجی» وجود دارد؛ مثلاً اگر مدل ابتدا در بخش *SELECT* یا *WHERE* انتخاب ضعیفی داشته باشد، در مراحل بعدی با توجه به توکن‌های جدید می‌تواند تصمیم بهتر بگیرد. محدودیت مهم آن، نیاز به تنظیم دقیق طول خروجی و برنامه زمان‌بندی پرشدن ماسک‌ها است؛ اگر این تنظیمات مناسب نباشد، خروجی ممکن است ناقص یا دارای بخش‌های اضافه باشد. شکل ۷ تفاوت شهودی این دو سبک را نشان می‌دهد.

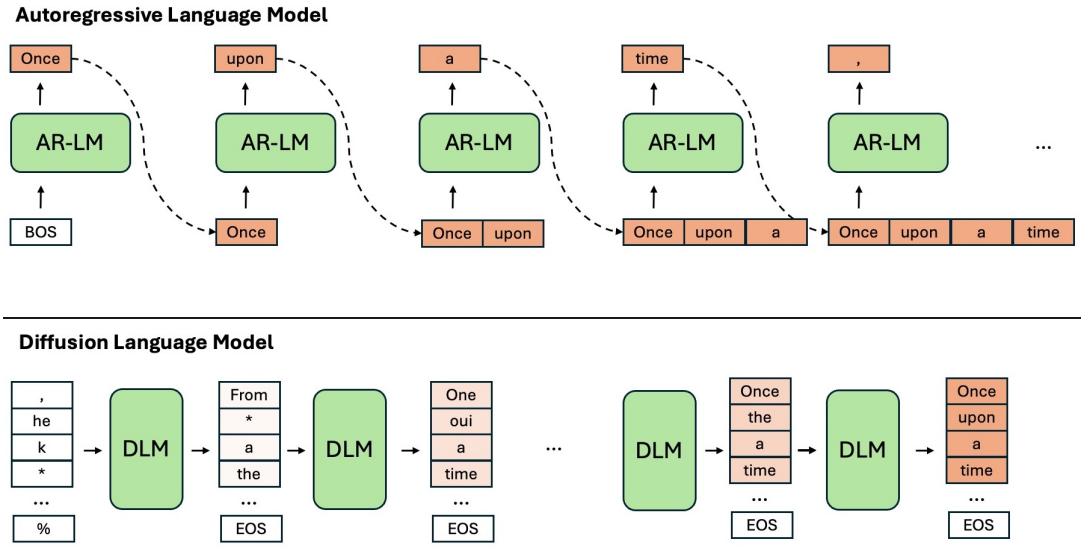
### ۲–۱–۲ *forward masking* به‌عنوان *noise model*

فرض کنید توالی تمیز $x = (x_1, \ldots, x_L)$ داریم. برای هر موقعیت $i$ یک متغیر برنولی نمونه‌گیری می‌کنیم:

$$m_i \sim \text{Bernoulli}(p)$$

اگر $m_i = 1$ باشد، $x_i$ با `[MASK]` جایگزین می‌شود وگرنه همان توکن اصلی نگه داشته می‌شود. بنابراین نسخه نویزی توالی برابر است با:

$$\tilde{x}_i = \begin{cases} \texttt{[MASK]} & m_i = 1 \\ x_i & m_i = 0 \end{cases}$$




شکل ۷: مقایسه شهودی تولید *autoregressive* و *masked iterative*

در پیاده‌سازی این پروژه، $p$ ثابت نیست و برای هر نمونه به‌صورت تصادفی در بازه $[0.05, 0.95]$ انتخاب می‌شود. دلیل این کار این است که مدل هم حالت «کم‌نویز» (اطلاعات زیاد) و هم حالت «پرنویز» (اطلاعات کم) را یاد بگیرد. اگر همیشه یک مقدار ثابت مثل $15\%$ استفاده شود، مدل فقط روی یک سطح خاص از ابهام آموزش می‌بیند و در زمان تولیدِ مرحله‌ای، روی وضعیت‌های خیلی کم‌ماسک یا خیلی پرماسک ضعیف می‌شود.

### ۲–۱–۳ چرا *loss* باید *reweight* شود

اگر *reweighting* انجام نشود، نمونه‌هایی که $p$ کوچک‌تری دارند توکن ماسک‌شده کمتری تولید می‌کنند، پس سهم آن‌ها در *loss* کل کمتر می‌شود. در نتیجه مدل ناخواسته بیشتر از نمونه‌های با ماسک زیاد یاد می‌گیرد و تعادل بین رژیم‌های مختلف نویز به‌هم می‌خورد. برای حل این مشکل، در کد از وزن تقریبی $w = \frac{1}{p}$ (با *clamp* تا سقف ۱۵) استفاده شد تا اثر نمونه‌های کم‌ماسک هم حفظ شود و مدل در کل طیف نویز پایدارتر آموزش ببیند.

## ۲–۲ داده، پرامپت و ابزارهای ارزیابی

### ۲–۲–۱ ساخت *pipeline* داده

از دیتاست *gretelai/synthetic_text_to_sql* استفاده شد و پس از *shuffle*، مطابق پیاده‌سازی نوت‌بوک تقسیم زیر ساخته شد:

| Size | Split |
|---|---|
| ۱۰۰۰ | Train |
| ۳۰۰ | Validation |
| ۳۰۰ | Test |

جدول ۱: اندازه داده‌های استفاده‌شده در این پیاده‌سازی

میانگین طول *question* در داده آموزشی برابر 83.335 و میانگین طول *schema* برابر 281.743 گزارش شد (بر حسب کاراکتر).

برای مدیریت *schema*های طولانی، از راهکار ساده و قابل‌تکرار زیر استفاده شد: تنظیم *MAX_LEN=512* و فعال بودن *truncation=True* در *tokenizer*. همچنین یک کنترل مهم اعمال شد تا فقط نمونه‌هایی نگه داشته شوند که در آن‌ها واقعاً بخشی از پاسخ در توالی نهایی حضور دارد:

```
prompt_len > len(input_ids)
```

در غیر این‌صورت نمونه حذف می‌شود تا مدل روی داده‌ای آموزش نبیند که پاسخ آن عملاً بریده شده است.

### ۲–۲–۲ طراحی *prompt* در قالب *chat format*

در آموزش از سه پیام استفاده شد:

- *system*: *"You are a Text-to-SQL assistant. Output ONLY the SQL query."*
- *user*: شامل *Schema* و *Question*
- *assistant*: شامل *gold SQL*

برای ساخت دقیق متن ورودی مدل، از *tokenizer.apply_chat_template* استفاده شد. برای تعیین مرز پاسخ، دو نسخه ساخته شد: (۱) نسخه کاملِ *system+user+assistant(sql)* و (۲) نسخه *prompt* بدون محتوای *assistant* ولی با *add_generation_prompt=True*. طول نسخه دوم به‌عنوان *prompt_len* ذخیره شد تا در زمان نویزی‌سازی فقط بخش پاسخ ماسک شود.

اهمیت پیام *system* و دستور *Output ONLY SQL* در *Exact Match* این است که هر متن اضافی (توضیح، جمله طبیعی، یا توکن‌های متا) باعث نابرابری متنی با پاسخ مرجع می‌شود؛ بنابراین محدودکردن خروجی به SQL خالص برای ارزیابی ضروری است.

### ۲–۲–۳ *SQL normalization* و معیارها

تابع *normalize_sql* در نوت‌بوک عملیات زیر را انجام می‌دهد: حذف کاراکترهای غیر ASCII، یکدست‌سازی حروف به *lowercase*، حذف *quotes/backticks*، حذف *semicolon* انتهایی، و نرمال‌سازی فاصله‌ها.

دو معیار استفاده‌شده:

- **EM Raw**: مقایسه مستقیم رشته خام پیش‌بینی با مرجع.
- **EM Normalized**: مقایسه پس از *normalization*.

محدودیت این معیارها: دو کوئری می‌توانند از نظر معنا معادل باشند ولی از نظر متن متفاوت باشند (مثلاً تفاوت در ترتیب شرط‌ها یا *alias*) و در *EM* اشتباه حساب شوند.

## ۲–۳ بارگذاری مدل و آموزش

### ۲–۳–۱ بارگذاری مدل و مفاهیم جدید

*4-bit quantization*: وزن‌های مدل به‌جای دقت بالا (مثلاً ۱۶ بیت)، با نمایش ۴ بیتی نگهداری می‌شوند تا حافظه مصرفی و هزینه بارگذاری کاهش یابد. در این پروژه از *NF4* و *double quantization* استفاده شده است.

*LoRA*: به‌جای آموزش همه پارامترهای مدل، چند ماتریس کم‌رتبه قابل‌آموزش روی لایه‌های هدف (مثل *q/k/v/o_proj*) اضافه می‌شود. این کار هزینه حافظه و زمان *fine-tuning* را خیلی کم می‌کند و برای GPU محدود مناسب است.

*KV cache*: در تولید *autoregressive*، *KV cache* محاسبات گذشته را نگه می‌دارد تا سرعت زیاد شود. اما در تولید تکراریِ ماسکی، در هر مرحله بخش‌های مختلف توالی تغییر می‌کنند و فرض پیوستگی چپ‌به‌راست برقرار نیست؛ بنابراین استفاده از کش کلاسیک مزیت محدودی دارد و در این پروژه غیرفعال شده است.

### ۲–۳–۲ پیاده‌سازی *forward masking*

تابع *apply_masking* یک *batch* ورودی را می‌گیرد و برای هر نمونه یک $p$ تصادفی در بازه $[0.05, 0.95]$ می‌سازد. سپس فقط در ناحیه پاسخ، توکن‌های غیر *pad/eos* با احتمال $p$ ماسک می‌شوند. خروجی تابع شامل:

- *noisy_ids*: توالی نویزی
- *labels*: فقط روی محل‌های ماسک‌شده مقدار واقعی دارد و بقیه *-100*
- *p_mask*: احتمال ماسک هر نمونه (برای *reweight*)

در *edge case* اگر به‌صورت تصادفی هیچ توکنی ماسک نشود، حداقل یک موقعیت ماسک می‌شود تا *loss* تهی یا *NaN* نشود.

### ۲–۳–۳ مرز *prompt/answer* و ماسک‌کردن پاسخ

در داده‌سازی، برای هر نمونه هم *input_ids* و هم *prompt_len* ذخیره شد. در تابع *apply_masking* فقط بازه [: prompt_len] قابل ماسک‌شدن است؛ یعنی *prompt* همیشه تمیز می‌ماند و فقط بخش پاسخ نویزی می‌شود.

دلیل این تصمیم: *prompt* نقش «شرط مسئله» (schema+question) را دارد. اگر prompt هم نویزی شود، مدل به‌جای یادگیری تولید SQL شرطی، بخشی از ظرفیت خود را صرف بازسازی ورودی می‌کند و هدف *Text-to-SQL* تضعیف می‌شود.

### ۲—۳—۴ *training loop* و *loss*

*loss* فقط روی توکن‌های ماسک‌شده پاسخ حساب می‌شود. در پیاده‌سازی:

- *cross-entropy* در سطح توکن محاسبه می‌شود.
- فقط مکان‌های *labels != -100!* وارد *loss* می‌شوند.
- وزن‌دهی با $w = \frac{1}{p}$ و *clamp(max=15)* اعمال می‌شود.

روش نرمال‌سازی انتخاب‌شده: تقسیم مجموع *loss* بر تعداد واقعی توکن‌های ماسک‌شده. این روش تفسیر ساده‌تری دارد (میانگین خطا روی همان توکن‌هایی که باید بازسازی شوند)، هرچند ممکن است با تغییر تعداد ماسک، نوسان مقیاس ایجاد شود؛ *reweighting* تا حدی این اثر را کاهش می‌دهد.

در مورد *padding*: توکن‌های *pad/eos* در ناحیه قابل ماسک حذف شده‌اند و در *labels* مقدار *100-* می‌گیرند؛ بنابراین عملاً وارد *loss* نمی‌شوند و تمرکز آموزش روی محتوای واقعی پاسخ است.

ابرپارامترهای اصلی:

- *EPOCHS=1* ،*GRAD_ACCUM=4* ،*BATCH_SIZE=1*
- *warmup=10%* با *cosine scheduler* ،*AdamW* ،*LR=1e-4*
- *gradient clipping=1.0*

تجمیع گرادیان باعث شبیه‌سازی *batch* مؤثر بزرگ‌تر با حافظه کم می‌شود، *scheduler* پایداری یادگیری را بهتر می‌کند، و *gradient clipping* جلوی انفجار گرادیان را می‌گیرد.

## ۲—۴ تولید متن و ارزیابی

### ۲—۴—۱ پیاده‌سازی *block diffusion sampling*

در تابع *generate_sql* ابتدا *prompt* ساخته می‌شود، سپس به طول *gen_len=96* توکن `[MASK]` به انتهای آن اضافه می‌گردد. بازه پاسخ به *block*های طول *block_len=12* تقسیم شده و برای هر *block* چند گام پالایش (*steps_per_block=8*) انجام می‌شود.

در هر گام:

۱. مدل روی توالی فعلی اجرا می‌شود و توزیع احتمال می‌دهد.

۲. برای هر موقعیت ماسک‌شده، بهترین توکن و *confidence* (احتمال *softmax* همان توکن) محاسبه می‌شود.

۳. فقط درصدی از موقعیت‌های با اطمینان بالاتر *commit* می‌شوند (حدود ۳۰٪ از ماسک‌های باقیمانده).

این سیاست بین سرعت و کیفیت تعادل ایجاد می‌کند: *commit* کم، فرصت اصلاح بیشتر می‌دهد؛ *commit* زیاد، تولید را سریع‌تر می‌کند.

### ۲–۴–۲ *post-processing* برای استخراج SQL

پس از تولید، متن خروجی تمیز می‌شود: حذف واژه‌های اضافه مثل *assistant/user*، حذف تکرارهای غیرعادی، و برش در اولین *semicolon*. دلیل نیاز به این مرحله این است که مدل‌های مکالمه‌ای ممکن است خروجی فرامتنی یا *junk tokens* تولید کنند که جزو SQL نیست.

### ۲–۴–۳ *evaluation* و تحلیل خطا

از آنجایی که تولید متن برای هر ۳۰۰ داده تست کاری بسیار زمان بر بود، ما این کار را تنها برای ۲۰ کوئری انجام دادیم.

در این اجرا که همراه گزارش ارسال شده است. نتیجه *Normalized EM* برابر:

$$\text{Accuracy} = \frac{6}{20} = 30\%$$

**خطاهای رایج مشاهده‌شده در این تست‌ها :**

- حذف شرط‌های ضروری یا استفاده از شرط اشتباه
- *JOIN* ناقص یا غلط
- خطاهای نحوی (تکرار *FROM*، نام جدول/ستون نادرست)
- سختی در کوئری‌های تحلیلی (*window functions*، *distinct*، *date filters*)

### ۲–۴–۴ پنج ایده برای بهبود

**ایده ۱ : *Curriculum* روی شدت ماسک** ابتدا با *p*های کوچک‌تر شروع شود و به‌تدریج به بازه کامل برسد. این کار می‌تواند همگرایی اولیه را پایدارتر کند و بعد مدل را برای شرایط سخت‌تر آماده کند.

**ایده ۲ : *Schema-aware truncation*** به‌جای بریدن ساده از انتها، ابتدا نام جدول‌ها و ستون‌های مرتبط با سؤال استخراج/نگه‌داری شوند و متن‌های توضیحی حذف شوند. انتظار می‌رود دقت در کوئری‌های *JOIN* بهتر شود.

**ایده ۳: *Execution-guided reranking*** برای هر سؤال چند کاندید SQL تولید شود و با اجرای سبک (یا بررسی نحوی/اجرایی) بهترین انتخاب شود. این روش می‌تواند خطاهای نحوی را کم و *EM* را افزایش دهد.

**ایده ۴: *Adaptive commit schedule*** به‌جای نرخ ثابت ۳۰٪، در گام‌های اول *commit* کم و در گام‌های آخر *commit* بیشتر شود. این کار فرصت اصلاح زودهنگام را بالا می‌برد و در پایان سرعت تکمیل را بیشتر می‌کند.

**ایده ۵: ترکیب *token-level* با *structure-level loss*** علاوه بر *cross-entropy* توکنی، یک جریمه ساختاری سبک برای الگوهای SQL (مثل حضور *SELECT-FROM* یا سازگاری پرانتزها) اضافه شود. این ایده می‌تواند پایداری نحوی خروجی را بهبود دهد.